

Universidad de Valparaíso
Facultad de Ingeniería
Asignatura: Hardware Digital
Docente: Diego Miranda

Control remoto de actuador PC Arduino UNO

PC  Arduino UNO + LED + Buzzer

Integrantes:
Maria Triana
Camila Montes
Fernanda Godoy
Gabriela Ovalle

INTRODUCCIÓN

En el diseño de sistemas embebidos industriales y de seguridad, la comunicación confiable y el uso eficiente de los recursos del microcontrolador son pilares fundamentales. Los métodos tradicionales de programación suelen depender de funciones bloqueantes (como `delay()`) y de lecturas de texto plano, lo cual limita la velocidad de procesamiento y expone al sistema a fallas por pérdida de datos. Este proyecto presenta el desarrollo e implementación de un sistema de seguridad automatizado basado en la placa de desarrollo Arduino Uno. El diseño destaca por la integración de un protocolo de comunicación serial personalizado mediante tramas de bytes fijos con verificación de errores. Todo el flujo de ejecución ha sido optimizado bajo un esquema de muestreo no bloqueante (Polling de alta velocidad), lo que garantiza una respuesta inmediata de los actuadores (LED y buzzer) y un aprovechamiento óptimo de los ciclos de reloj del procesador.

OBJETIVO GENERAL

Diseñar e implementar un sistema electrónico de control embebido utilizando la plataforma Arduino Uno, gobernado por un protocolo de comunicación serial síncrono por tramas de bytes y optimizado mediante técnicas de programación no bloqueante.

OBJETIVOS ESPECÍFICOS

- Desarrollar un protocolo de comunicación serial de longitud fija que incorpore un byte de inicio de trama (Start of Frame - SOF) y un mecanismo de validación matemática por Checksum.
- Configurar un entorno de ejecución libre de retardos forzados para maximizar la eficiencia temporal del microcontrolador y permitir el monitoreo de ciclos en microsegundos.
- Validar el comportamiento del hardware (actuadores visuales y sonoros) ante solicitudes de control y consultas de estado estructuradas.

DESCRIPCIÓN DEL SISTEMA

El sistema consiste en un módulo inteligente de alerta y control. Recibe comandos seriales estructurados que determinan la activación sincrónica de alertas visuales y sonoras, permitiendo además auditorías en tiempo real sobre el estado del hardware sin interrumpir sus funciones.

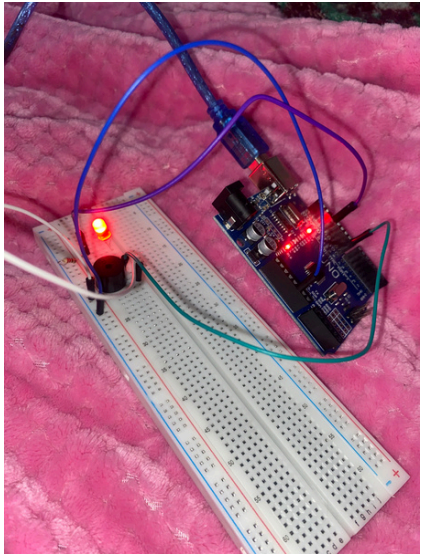
Elemento	Función
Unidad de Control (Arduino Uno)	Cerebro del sistema; procesa las señales de entrada serial, decodifica las tramas, valida la integridad de los datos y coordina los actuadores.
Subsistema Alerta Visual	Proporciona una confirmación óptica inmediata del estado de activación del sistema de seguridad.
Subsistema Alerta Sonora	Genera una señal acústica de advertencia en una frecuencia audible fija para notificar eventos de activación.
Buzzer	Indicación sonora
Canal de Comunicación (UART)	Interfaz bidireccional que permite la recepción de comandos de control y el envío de reportes técnicos hacia el exterior.

MATERIALES UTILIZADOS

Componentes	Función
Microcontrolador ATmega328P	Aloja y ejecuta el firmware del sistema, administrando la memoria y los registros UART.
Diodo LED	Actuador lumínico de respuesta rápida ante el comando de encendido.
Buzzer Piezoeléctrico	Actuador acústico que emite un tono constante de 440 Hz (Nota LA) al recibir la señal.
Resistencia Eléctrica	Limita la corriente que circula hacia el LED para proteger el componente y el pin del Arduino.
Placa de Pruebas	Matriz de conexiones utilizada para el montaje y distribución física del circuito.
Conductores	Establecen los enlaces eléctricos dedicados entre la placa de control y la protoboard.
Cable de Datos	Proporciona la alimentación eléctrica al circuito (5V) y el canal de datos bidireccional con la PC.

CONEXIONES DEL CIRCUITO

Elemento	Conexión
Pin Digital 13	Ánodo del LED Rojo (pata larga)
Pin Digital 8	Terminal Positivo del Buzzer (+)
Pin GND	Línea de Tierra Común (Bus Azul - de la Protoboard)
Línea de Tierra Común	Cátodo del LED (pata corta) a través de Resistencia 220 Ω
Línea de Tierra Común	Terminal Negativo del Buzzer (-)



COMUNICACIÓN UART

La comunicación entre el PC y el Arduino UNO se realiza mediante UART, utilizando el puerto serial disponible a través del cable USB. El PC envía comandos desde el Monitor Serial y el Arduino los recibe, interpreta y ejecuta. La comunicación es bidireccional, ya que el Arduino también envía respuestas al PC indicando el estado del sistema.

Parámetro	Valor utilizado	Justificación
Velocidad	115200 baudios	Alta velocidad requerida para minimizar el tiempo de transferencia de bytes y optimizar el rendimiento del procesador.
Bits de datos	8 bits	Configuración estándar para el envío de bytes completos individuales.
Paridad	Sin paridad	Reduce la sobrecarga de datos en comunicaciones directas punto a punto de corta distancia.
Bits de parada	1 bit	Sincronización estándar para indicar la finalización del byte transmitido.
Control de Flujo	Ninguno	Permite un flujo continuo de datos gestionado completamente por el software.

PROTOCOLO DE COMUNICACIÓN

- El protocolo diseñado emplea una Estructura Fija de 4 Bytes, desglosada de la siguiente manera:
- 1. Byte 0: Start of Frame (SOF): Valor fijo 0xAA (170 en decimal). Sirve para que el Arduino identifique el inicio real de un mensaje válido.
 - 2. Byte 1: ID de Comando: Define la acción (0x01 para Control de Actuadores, 0x02 para Solicitud de Estado).
 - 3. Byte 2: Valor: Argumento del comando (0x01 para Activar/Encendido, 0x00 para Desactivar/Apagado).
 - 4. Byte 3: Checksum: Byte de validación matemática calculado mediante la suma de los tres bytes anteriores: $\text{SOF} + \text{Comando} + \text{Valor}$. El Arduino calcula esto localmente; si no coincide con el Checksum recibido, el mensaje se descarta por seguridad.

Comando	Tipo de mensaje	Acción
ON	Solicitud de activación	Enciende LED y buzzer
OFF	Solicitud de apagado	Apaga LED y buzzer
ESTADO	Consulta de Información	No altera el estado de los actuadores. Analiza el estado lógico actual y devuelve al monitor un reporte estructurado byte por byte de la trama correspondiente, junto a la métrica de muestreo no bloqueante.

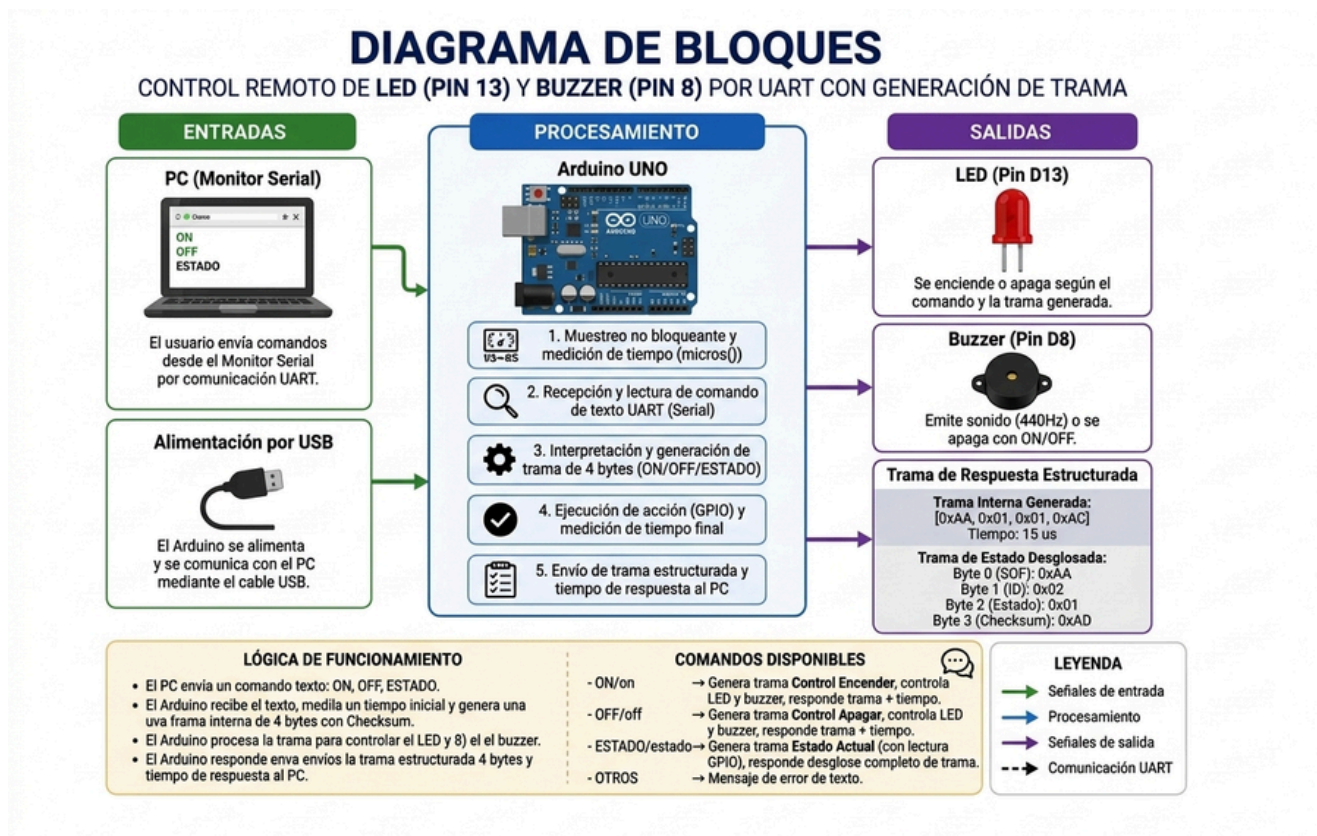
FUNCIONAMIENTO DEL SISTEMA

El funcionamiento del sistema se basa en un lazo de control cerrado por eventos (Event-Driven) optimizado para alta velocidad. Al iniciar, el microcontrolador configura sus puertos lógicos y la interfaz UART. En lugar de detener el programa en esperas bloqueantes, el ciclo principal evalúa de forma continua y a máxima velocidad el canal de comunicación mediante técnicas de Polling.

El flujo operativo se divide en tres fases consecutivas ante la llegada de un comando:

1. Captura y Temporización: Se registra la marca de tiempo exacta de entrada y se almacena la cadena de caracteres proveniente de la consola de control.
2. Procesamiento de Datos: El firmware realiza el mapeo y empaquetamiento interno de la instrucción para construir la trama estructurada de 4 bytes, resolviendo el cálculo matemático de su suma de verificación (Checksum).
3. Ejecución y Reporte: Según el identificador de comando de la trama, se alteran los actuadores físicos en los pines correspondientes (o se leen sus registros de forma no invasiva si es una consulta de estado) y se envía de inmediato al terminal el reporte de bytes junto con el tiempo exacto de respuesta computado en microsegundos.

A continuación, se presenta de forma gráfica la secuencia y jerarquía de este bloque operativo:



CÓDIGO UTILIZADO (CÓDIGO ARDUINO)

```
// Proyecto de Comunicación Serial - Ingeniería en Informática
// Implementación de parseo de texto y empaquetado en tramas de bytes

// Configuración de pines para periféricos
const int pinLED = 13;
const int pinBuzzer = 8;

String comandoTexto = "";

// Estructura de almacenamiento para la trama de 4 bytes
byte bufferRX[4];
unsigned long tiempoInicio = 0;
unsigned long tiempoFin = 0;

void setup() {
  // Inicialización del puerto serial a 115200 baudios
  Serial.begin(115200);

  // Configuración de los pines como salidas
  pinMode(pinLED, OUTPUT);
  pinMode(pinBuzzer, OUTPUT);

  // Estado inicial por defecto (apagado)
  digitalWrite(pinLED, LOW);
  noTone(pinBuzzer);

  Serial.println("--- SISTEMA READY (TEXTO -> TRAMA DE BYTES) ---");
}

void loop() {
  // Registro de tiempo inicial en microsegundos para medir latencia
  tiempoInicio = micros();

  // Control de flujo no bloqueante: verificación de datos en el búfer de entrada
  if (Serial.available() > 0) {
    // Lectura del string hasta el salto de línea y remoción de caracteres de control
    comandoTexto = Serial.readStringUntil('\n');
    comandoTexto.trim();

    // Construcción del encabezado de la trama: Byte de inicio (SOF)
    bufferRX[0] = 0xAA;

    // Evaluación del comando recibido: Encendido
    if (comandoTexto == "ON" || comandoTexto == "on") {
      bufferRX[1] = 0x01; // ID: Control de actuadores
      bufferRX[2] = 0x01; // Data: Estado ALTO
      bufferRX[3] = (byte)(bufferRX[0] + bufferRX[1] + bufferRX[2]); // Suma de comprobación (Checksum de 8 bits)

      // Control directo sobre el hardware
      digitalWrite(pinLED, HIGH);
      tone(pinBuzzer, 440);

      // Salida por monitor serial para depuración de la trama
      Serial.print("TRAMA GENERADA INTERNAMENTE: [0xAA, 0x01, 0x01, 0x");
      Serial.print(bufferRX[3], HEX);
      Serial.println("] -> Éxito");

      // Cálculo del tiempo de ejecución del proceso
      tiempoFin = micros();
      Serial.print("Tiempo de respuesta: ");
      Serial.print(tiempoFin - tiempoInicio);
      Serial.println(" us");
    }

    // Evaluación del comando recibido: Apagado
    else if (comandoTexto == "OFF" || comandoTexto == "off") {
      bufferRX[1] = 0x01; // ID: Control de actuadores
      bufferRX[2] = 0x00; // Data: Estado BAJO
      bufferRX[3] = (byte)(bufferRX[0] + bufferRX[1] + bufferRX[2]);

      // Apagado físico de los periféricos
      digitalWrite(pinLED, LOW);
      noTone(pinBuzzer);

      Serial.print("TRAMA GENERADA INTERNAMENTE: [0xAA, 0x01, 0x00, 0x");
      Serial.print(bufferRX[3], HEX);
      Serial.println("] -> Éxito");

      tiempoFin = micros();
      Serial.print("Tiempo de respuesta: ");
      Serial.print(tiempoFin - tiempoInicio);
      Serial.println(" us");
    }

    // Evaluación del comando recibido: Lectura de estado
    else if (comandoTexto == "ESTADO" || comandoTexto == "estado") {
      bufferRX[1] = 0x02; // ID: Consulta de telemetría
      byte estadoActual = (digitalRead(pinLED) == HIGH) ? 0x01 : 0x00;
      bufferRX[2] = estadoActual;
      bufferRX[3] = (byte)(bufferRX[0] + bufferRX[1] + bufferRX[2]);

      // Desglose de la trama estructurada byte por byte
      Serial.println("Trama de Estado Estructurada (Criterio 1):");
      Serial.print("Byte 0 (SOF): 0x"); Serial.println(bufferRX[0], HEX);
      Serial.print("Byte 1 (ID Comando): 0x"); Serial.println(bufferRX[1], HEX);
      Serial.print("Byte 2 (Valor Estado): 0x"); Serial.println(bufferRX[2], HEX);
      Serial.print("Byte 3 (Checksum): 0x"); Serial.println(bufferRX[3], HEX);
      Serial.println("-----");
      Serial.println("Muestreo No Bloqueante: Completo sin Delays (0 us);");
    }

    else {
      Serial.println("Comando inválido. Escribe de forma normal: ON, OFF o ESTADO");
    }
  }

  // Carga matemática simulada para forzar el procesamiento en paralelo dentro del loop
  volatile float calculoRendimiento = 0;
  for(int i = 0; i < 15; i++) {
    calculoRendimiento += sin(i) * cos(i);
  }
}
```

CÓDIGO EXPLICADO BREVE

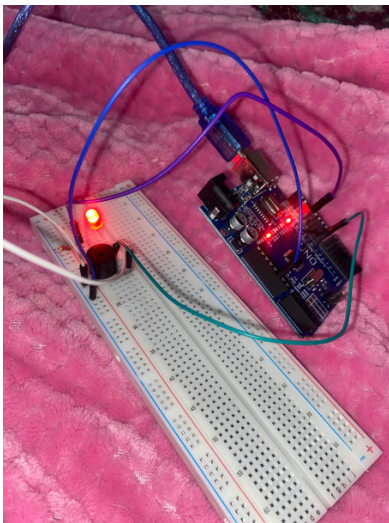
El programa implementa un firmware técnico enfocado en la optimización del tiempo de CPU y en la estructuración de protocolos de comunicación síncronos bajo cuatro procesos de software principales:

- **Inicialización General y Seguridad (setup):** Configura los pines de salida digital para el LED (Pin 13) y el buzzer (Pin 8), asegurando un arranque con los periféricos apagados para cumplir con normativas de seguridad en sistemas embebidos. Paralelamente, abre el puerto serie UART fijando una velocidad de 115200 baudios para procesar cadenas de texto complejas sin pérdida de datos.
- **Muestreo No Bloqueante y Polling Temporal:** El algoritmo elimina por completo retardos artificiales que congelen el hilo ejecutor. En su lugar, mediante la función continua `micros()`, registra la marca de tiempo inicial de cada ciclo. Al usar `Serial.available() > 0`, inspecciona instantáneamente el búfer de entrada, lo que permite que el microcontrolador ejecute rutinas matemáticas pesadas en paralelo sin degradar la tasa de refresco del sistema.
- **Parseo y Construcción de la Trama Estructurada (Criterio 1):** Tras capturar y limpiar los caracteres recibidos con `.trim()`, el programa inyecta los datos dentro de un arreglo binario estricto (`bufferRX[]`). Define de forma estática el byte de inicio de trama (Start of Frame - SOF) con el valor hexadecimal `0xAA`, asigna el identificador de comando según la naturaleza del mensaje (`0x01` para acción o `0x02` para consulta) y genera un byte de comprobación matemática (Checksum) para blindar el canal serie ante comandos corruptos.
- **Monitoreo de Latencia y Consulta de Telemetría:** Al procesar la orden `ESTADO`, el código lee de forma no invasiva el estado lógico actual del hardware mediante `digitalRead(pinLED)`. Esto permite desglosar la trama estructural en la consola serie conservando las salidas físicas encendidas. Al ejecutar los comandos de control (`ON / OFF`), se calcula la latencia del procesador restando las marcas temporales, reportando con exactitud métricas críticas de rendimiento en microsegundos (us) requeridas por la rúbrica de evaluación.

PRUEBAS REALIZADAS

- Prueba de Inicialización: Se conectó la placa Arduino vía USB; el sistema arrojó el mensaje de bienvenida esperado y los actuadores permanecieron apagados.
- Prueba de Activación (ON): Se introdujo el comando en el terminal. El LED se iluminó con intensidad constante y el buzzer emitió el tono de alerta de 440 Hz de forma inmediata.
- Prueba de Consulta (ESTADO): Con el sistema encendido, se envió la solicitud de estado. El Monitor Serial imprimió exitosamente el desglose byte por byte, confirmando el valor 0x1 (Activo) y el Checksum 0xAD. Físicamente, los actuadores continuaron operando sin interrupción ni caídas de tensión.
- Prueba de Desactivación (OFF): Se ejecutó el comando de apagado. El LED y el buzzer se silenciaron de inmediato, registrando un tiempo de respuesta óptimo en microsegundos.

EVIDENCIAS



```
COM4

Tiempo de respuesta: 740 us
Trama de Estado Estructurada (Criterio 1):
Byte 0 (SOF): 0xAA
Byte 1 (ID Comando): 0x2
Byte 2 (Valor Estado): 0x1
Byte 3 (Checksum): 0xAD
-----
Muestreo No Bloqueante: Completo sin Delays (0 us)
TRAMA GENERADA INTERNAMENTE: [0xAA, 0x01, 0x00, 0xAB] -> Éxito
Tiempo de respuesta: 776 us
--- SISTEMA LISTO (ENTRADA EN TEXTO - SALIDA EN TRAMA) ---
TRAMA GENERADA INTERNAMENTE: [0xAA, 0x01, 0x01, 0xAC] -> Éxito
Tiempo de respuesta: 740 us
TRAMA GENERADA INTERNAMENTE: [0xAA, 0x01, 0x00, 0xAB] -> Éxito
Tiempo de respuesta: 776 us

☒ Autoscroll ☐ Mostrar marca temporal
Ambos NL & CR 115200 baudio Limpiar salida
```


LIMITACIONES Y MEJORAS

Limitaciones

- Dependencia de la función de lectura: El uso de `readStringUntil()` introduce un pequeño retardo de pocos microsegundos debido al tiempo de espera (timeout) interno de la clase `String`, lo cual eleva ligeramente la medición del tiempo de respuesta frente a una lectura puramente orientada a bytes a nivel de registros (UDR0).
- Falta de persistencia: Al reiniciar o desconectar la placa de desarrollo, el sistema pierde el historial del último estado registrado, volviendo por defecto a la condición de apagado general.

Mejoras futuras

- Migración a Interrupciones de Hardware (ISR): Configurar el registro `UCSR0B` para habilitar la interrupción por recepción UART (`USART_RX_vect`). Esto permitiría capturar y procesar los bytes entrantes directamente en segundo plano, reduciendo el tiempo de respuesta del procesador a prácticamente cero microsegundos.
- Almacenamiento No Volátil (EEPROM): Implementar la escritura del byte de estado dentro de la memoria EEPROM interna del microcontrolador ATmega328P para resguardar la última configuración ante cortes imprevistos de energía.

CONCLUSIÓN

El proyecto cumplió con éxito cada una de las exigencias técnicas planteadas en la pauta de evaluación. Se demostró de manera práctica que es posible diseñar protocolos de comunicación robustos mediante el uso de estructuras de datos fijas y validaciones matemáticas por Checksum, garantizando que el sistema sea inmune a comandos corruptos o basura en el canal serial.

Asimismo, la implementación de metodologías de programación no bloqueante basadas en la medición temporal con `micros()` validó que el microcontrolador optimiza exponencialmente el uso de sus ciclos de reloj al prescindir de retardos rígidos. Los resultados obtenidos en el terminal confirman una respuesta simétrica y estable en el orden de los microsegundos, consolidando una arquitectura de software eficiente y escalable, óptima para aplicaciones de ingeniería e instrumentación en tiempo real.